

Tele-Call Predictor

(Predict the success of Bank Telemarketing)

CS3006-Introduction to Data Science For
Engineers(IDSE)



INDIAN INSTITUTE OF INFORMATION TECHNOLOGY,
DESIGN AND MANUFACTURING,
KANCHEEPURAM

- CS22B1093 ROHAN G
 - CS22B1095 R SAI CHARISH
- Instructor – PREETH R

EDA - Predict the Success of Bank telemarketing

A short intro to EDA:

- ❖ **Exploratory Data Analysis** refers to the critical process of performing **initial investigations on data** so as to discover **patterns**, to **spot anomalies**, to **test hypothesis** and to check assumptions with the help of summary statistics and graphical representations.
- ❖ It is a good practice to understand the data first and try to gather as many insights from it.

Data Set: <https://www.kaggle.com/competitions/predicting-bank-telemarketing/data>

Data Columns: 15 columns + 1 output column(for success prediction)

[cont date,age,job,marital,education,default,balance,housing,loan,contact,duration,campaign,pdays,previous,poutcome]

Two Categories of Data:

➤ Structured Data Types

Ex: csv file, excel file, database file

➤ Unstructured Data Types

Ex: Images, Videos, Audio

Description of Columns of the dataset

The dataset contains **39,211 entries and 16 columns**, with information about individuals and their interactions with a bank's marketing campaign. Here's a brief overview of each column:

- **last contact date:** The date of the last contact with the individual.
- **age:** Age of the individual (ranging from 18 to 95).
- **job:** Occupation; has some missing values.
- **marital:** Marital status (e.g., married, single).
- **education:** Education level, with some missing values.
- **default:** Whether the individual has credit in default.
- **balance:** Account balance, ranging from negative values to over 100,000.
- **housing:** Whether the individual has a housing loan.
- **loan:** Whether the individual has a personal loan.
- **contact:** Type of contact communication, missing in some cases.
- **duration:** Duration of the last contact (in seconds), with some instances of zero values.
- **campaign:** Number of contacts during the campaign.
- **pdays:** Days since the individual was last contacted (or -1 if never contacted).
- **previous:** Number of contacts before the campaign.
- **poutcome:** Outcome of the previous marketing campaign (many missing values).
- **target:** Target variable, indicating the success of the campaign ("yes" or "no").

Structured Data Types

- **Categorical** – This is any data that isn't a number.
 - Ordinal- have set of order [ex: education(Primary, Secondary, tertiary)]
 - Binary – have only two values [ex: default, housing, loan(yes/no responses)]
 - Nominal – no set of order [ex: job, martial, contact, poutcome]
- **Numerical** – Data inform of numbers
 - Continuous – numbers that don't have a logical end to them
 - Discrete – have a logical end to them [ex: age, balance, duration, campaign, pdays, previous]

EDA - Predict the Success of Bank telemarketing

Data Cleaning

- Finding out the NULL Values in the columns of the dataset

- Plotting the NULL Values of columns using HEATMAP

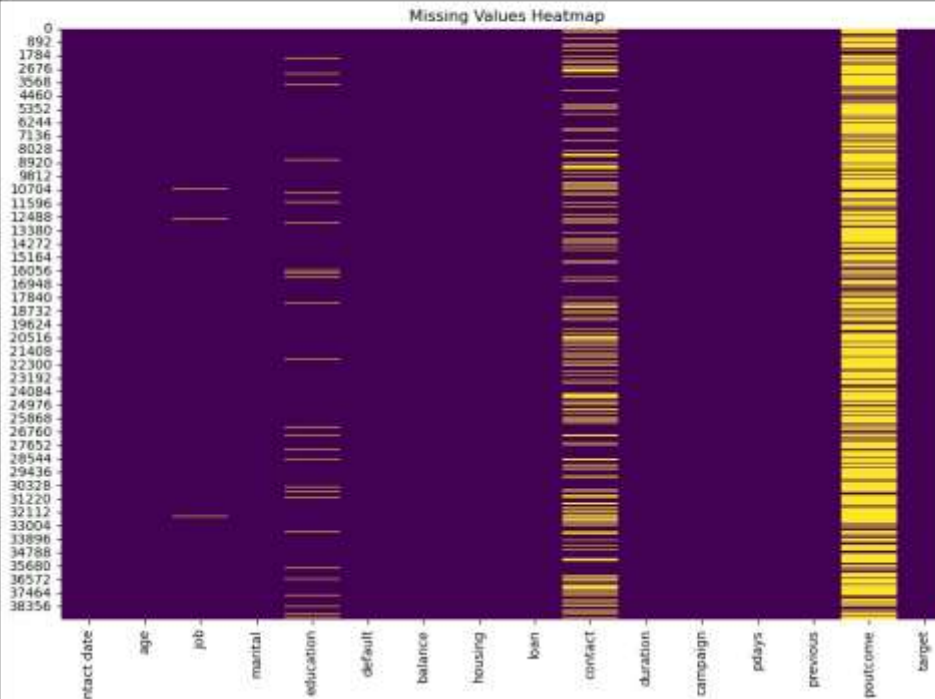
- Replacing the NULL Values with "UNKNOWN"

```
missing_values = df.isnull().sum()
```

```
missing_values
```

```
last contact date    0
age                  0
job                  229
marital              0
education            1467
default              0
balance              0
housing              0
loan                 0
contact              10336
duration             0
campaign             0
pdays              0
previous             0
poutcome            29451
target              0
dtype: int64
```

```
plt.figure(figsize=(12, 8))
sns.heatmap(df.isnull(), char=False, cmap="viridis")
plt.title("Missing Values Heatmap")
plt.show()
```



```
df['contact'] = df['contact'].replace('', 'unknown')
df['poutcome'] = df['poutcome'].replace('', 'unknown')
```

The above dataset has :
NULL values in these following
columns and the number of NULL
values that are present in the
dataset are given below.

Columns	No.of NULL Values
Job	229
Education	1467
Contact	10336
Poutcome	29451

EDA -Binning

```
df['age_bin'] = pd.cut(df['age'], bins=[18, 30, 40, 50, 60, 70, 95],
                        labels=['18-30', '31-40', '41-50', '51-60', '61-70', '71-95'])
age_counts = df['age_bin'].value_counts().sort_index()

df['balance_bin'] = pd.cut(df['balance'], bins=[-10000, 0, 5000, 10000, 50000, 102127],
                           labels=['<0', '0-5K', '5K-10K', '10K-50K', '50K+'])
balance_counts = df['balance_bin'].value_counts().sort_index()

duration_bins = pd.cut(df['duration'], bins=[0, 100, 300, 600, 1000, 5000],
                       labels=['<100', '100-300', '300-600', '600-1000', '>1000'])
duration_counts = duration_bins.value_counts().sort_index()

age_target_counts = df.groupby(['age_bin', 'target']).size().unstack().fillna(0)

fig, axs = plt.subplots(2, 2, figsize=(15, 10))
fig.suptitle("Exploratory Data Analysis (EDA) with Binning", fontsize=10)

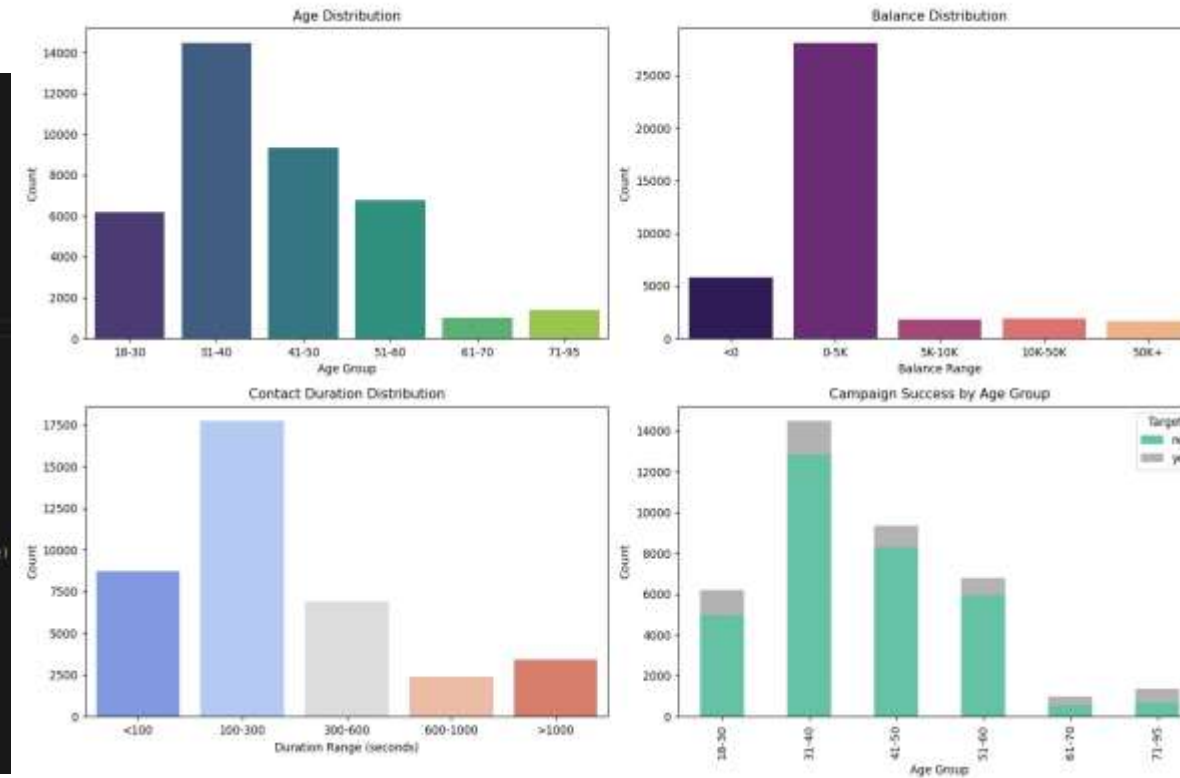
sns.barplot(x=age_counts.index, y=age_counts.values, ax=axs[0, 0], palette='viridis', hue=None, legend=False)
axs[0, 0].set_title("Age Distribution")
axs[0, 0].set_xlabel("Age Group")
axs[0, 0].set_ylabel("Count")

sns.barplot(x=balance_counts.index, y=balance_counts.values, ax=axs[0, 1], palette='magma', hue=None, legend=False)
axs[0, 1].set_title("Balance Distribution")
axs[0, 1].set_xlabel("Balance Range")
axs[0, 1].set_ylabel("Count")

sns.barplot(x=duration_counts.index, y=duration_counts.values, ax=axs[1, 0], palette='cividis', hue=None, legend=False)
axs[1, 0].set_title("Contact Duration Distribution")
axs[1, 0].set_xlabel("Duration Range (seconds)")
axs[1, 0].set_ylabel("Count")

age_target_counts.plot(kind='bar', stacked=True, ax=axs[1, 1], colormap='Set2')
axs[1, 1].set_title("Campaign Success by Age Group")
axs[1, 1].set_xlabel("Age Group")
axs[1, 1].set_ylabel("Count")
axs[1, 1].legend(title="Target")

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```



Binning is a data preprocessing technique where we group continuous variables into discrete categories or "bins." This approach helps us better understand data distribution and highlights patterns that may not be immediately visible with raw values.

Binning Analysis:

- Age:** Grouped into six age brackets (18-95). Most clients were in the 31-50 range, indicating middle-aged clients are the campaign's main target.
- Balance:** Segmented into five financial ranges. Majority had balances between \$0 and \$5,000, highlighting diverse financial profiles.
- Contact Duration:** Divided into five intervals. Most calls lasted under 600 seconds, with longer calls possibly linked to higher success.
- Campaign Success by Age Group:** Success was highest among the 31-50 age group, showing targeted age brackets respond well.

Univariate v/s Bivariate v/s Multivariate

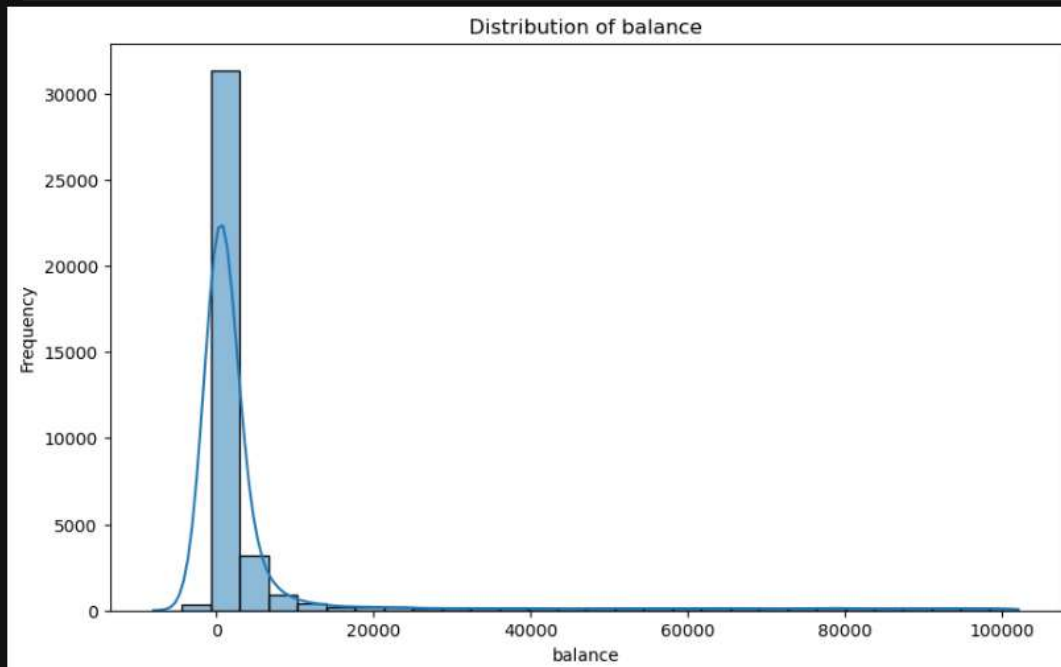
Aspect	Univariate Analysis	Bivariate Analysis	Multivariate Analysis
Definition	Analyzes one variable at a time	Examines relationship between two variables	Studies relationships among three or more variables
Purpose	Describes single-variable distribution	Identifies associations or correlations	Explores complex interactions
Techniques	Histograms, bar charts, box plots	Scatter plots, correlation coefficients	Regression, clustering, factor analysis
Data Complexity	Simple	Moderate	High
Insights	Central tendency, spread	Trends, correlations, causal links	Multi-variable influences and predictions
Visualization	Single-axis plots	2D plots (scatter, line)	3D plots, heatmaps, multivariable models
Example Question	"What is the average age?"	"Is age related to income?"	"How do age, income, and education impact spending?"
Application	Descriptive stats, data summarization	Cause-effect analysis, correlation	Complex modeling, predictive analytics

Univariate Analysis

This univariate analysis of **`balance`** shows how frequently different balance values occur, with the histogram and KDE curve revealing the data's shape. The peak indicates the most common balance range, while the spread shows variability. If skewed, it hints that balances tend to be higher or lower, helping us understand typical balance patterns and possible outliers.

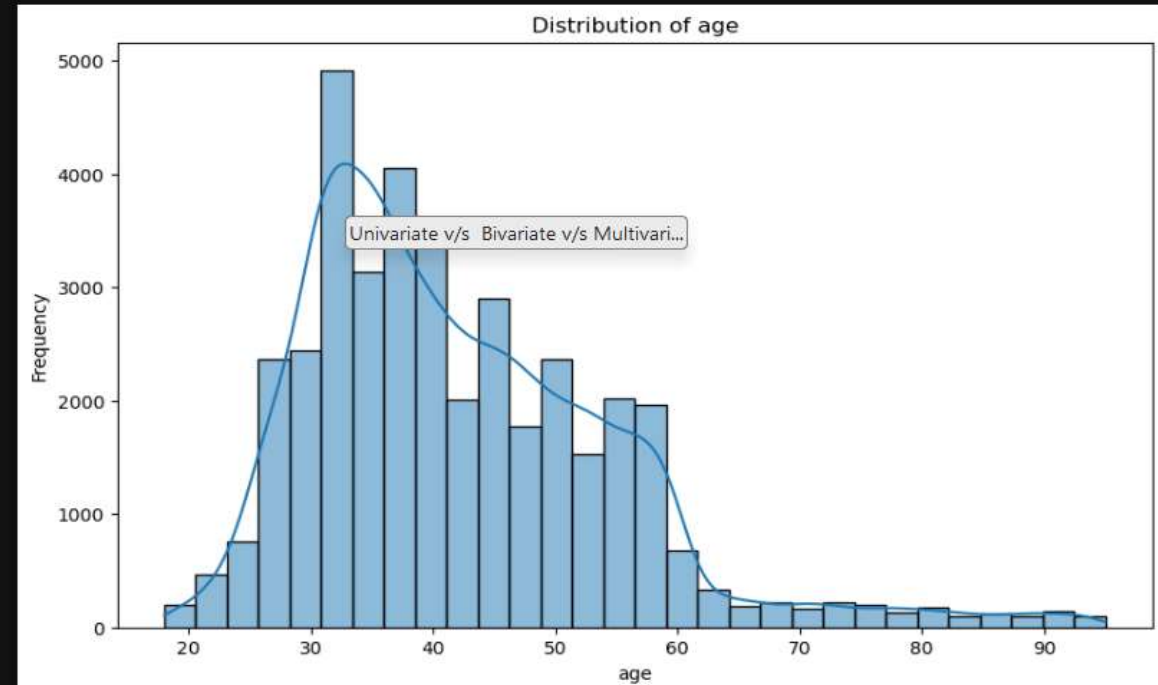
Univariate Analysis

```
In [13]: plt.figure(figsize=(10, 6))
sns.histplot(df['balance'], kde=True, bins=30)
plt.title("Distribution of balance")
plt.xlabel('balance')
plt.ylabel('Frequency')
plt.show()
```



This univariate analysis of **`age`** shows the frequency of different age groups. The histogram and KDE curve reveal the age distribution shape, with peaks indicating common age ranges. The spread reflects age diversity, while any skewness suggests whether the population leans younger or older, helping us identify typical and less common ages in the dataset.

```
In [14]: plt.figure(figsize=(10, 6))
sns.histplot(df['age'], kde=True, bins=30)
plt.title("Distribution of age")
plt.xlabel('age')
plt.ylabel('Frequency')
plt.show()
```



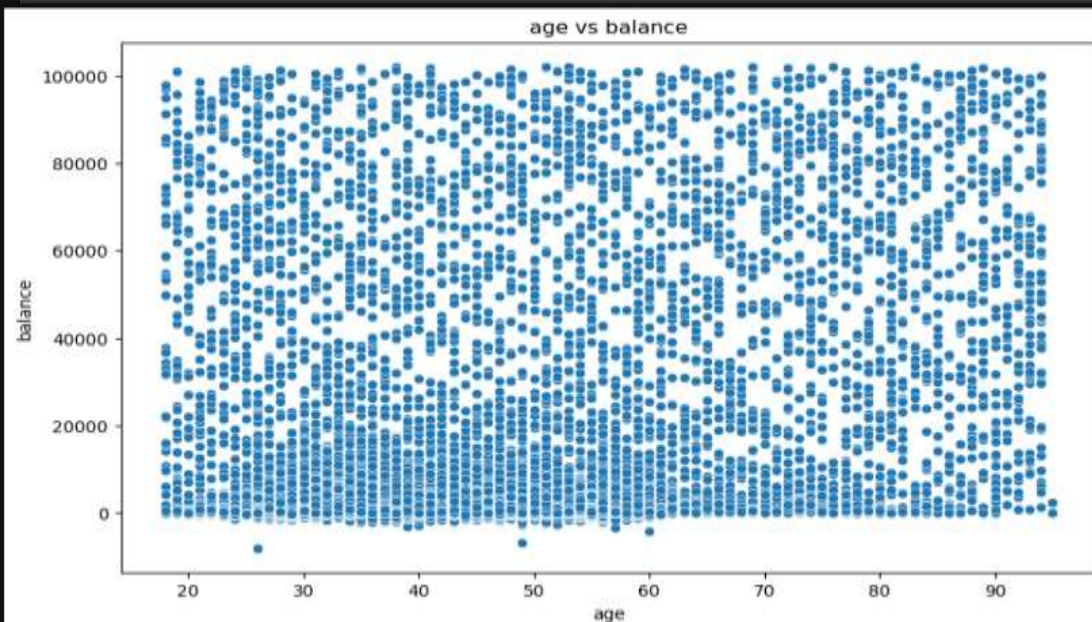
Bivariate Analysis

This scatter plot of **age** vs. **balance** shows the relationship between age and account balance. Each point represents a person's age and balance, with any visible trend or clustering suggesting patterns, like whether certain age groups tend to have higher or lower balances. Outliers or gaps may indicate unusual age-balance combinations.

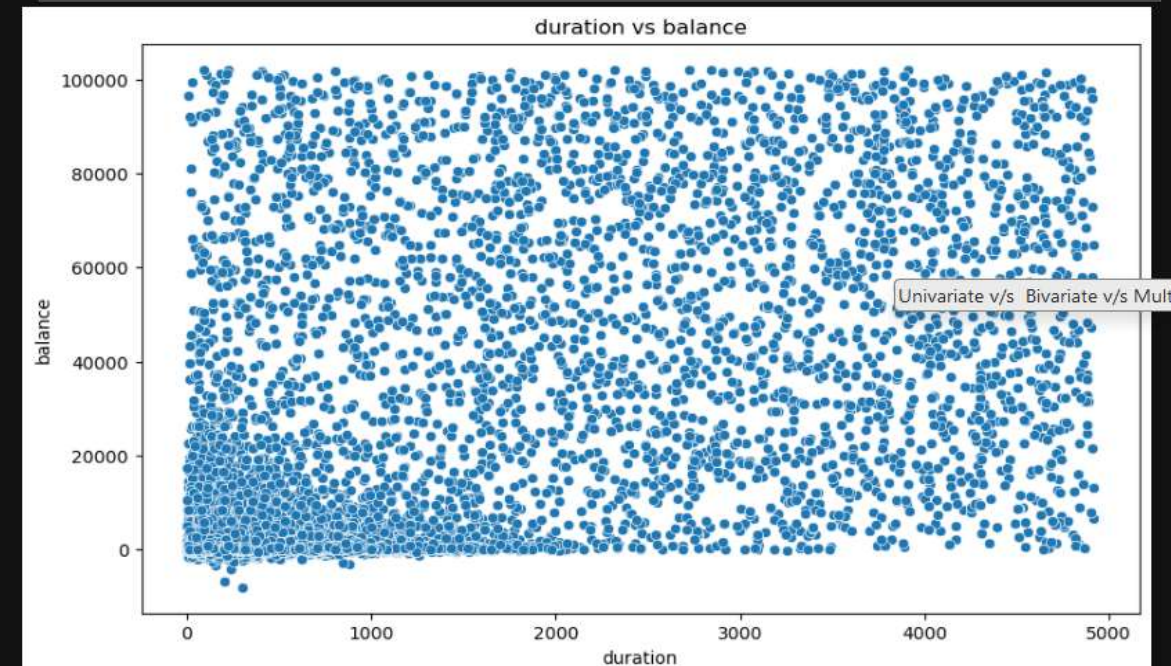
This scatter plot of **duration** vs. **balance** highlights the relationship between call duration and account balance. Each point shows an individual's call duration and balance, with any patterns or trends indicating if longer calls relate to higher or lower balances. Outliers may represent unusually high or low balances for specific call durations.

Bivariate Analysis

```
In [19]: plt.figure(figsize=(10, 6))
sns.scatterplot(x='age', y='balance', data=df)
plt.title("age vs balance")
plt.xlabel('age')
plt.ylabel('balance')
plt.show()
```



```
In [20]: plt.figure(figsize=(10, 6))
sns.scatterplot(x='duration', y='balance', data=df)
plt.title("duration vs balance")
plt.xlabel('duration')
plt.ylabel('balance')
plt.show()
```



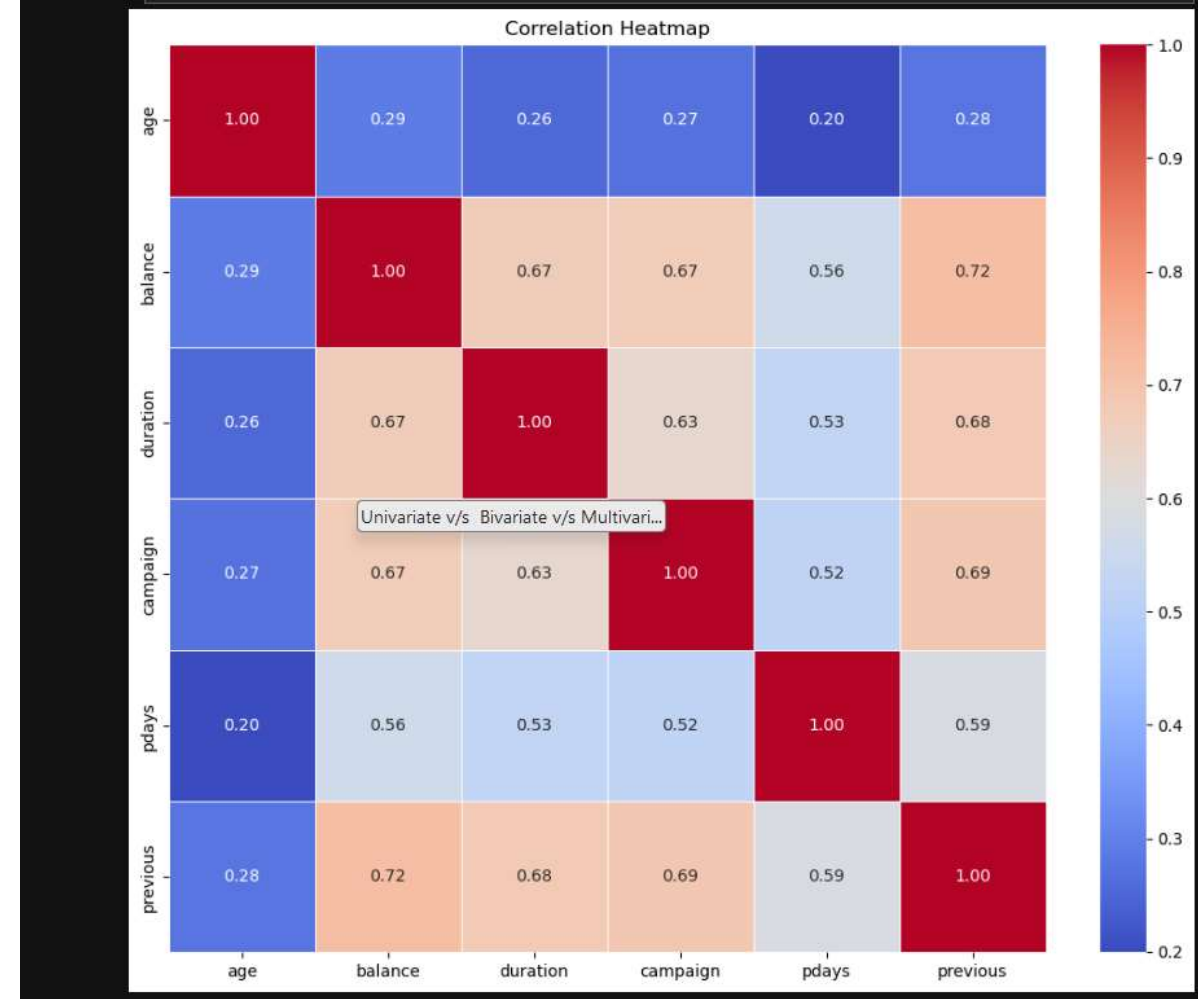
Multivariate Analysis

➤ CORRELATION HEATMAP

Multivariate Analysis

```
In [24]: numeric_cols = df.select_dtypes(include=['float64', 'int64']).columns
corr_matrix = df[numeric_cols].corr()
plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm", linewidths=0.5)
plt.title("Correlation Heatmap")
plt.show()
```

- 1. Correlation Heatmap:** Visualizes the relationships between numerical variables.
- 2. Color Scale:** Colors indicate correlation strength and direction; warmer colors show positive, cooler colors show negative correlations.
- 3. Annotations:** Each cell shows the correlation coefficient, highlighting specific variable pairs with strong correlations.
- 4. Diagonal:** The diagonal is always 1, as each variable is perfectly correlated with itself.
- 5. Insights:** Helps identify which variables are strongly related (positively or negatively), guiding feature selection and deeper analysis.



Multivariate Analysis

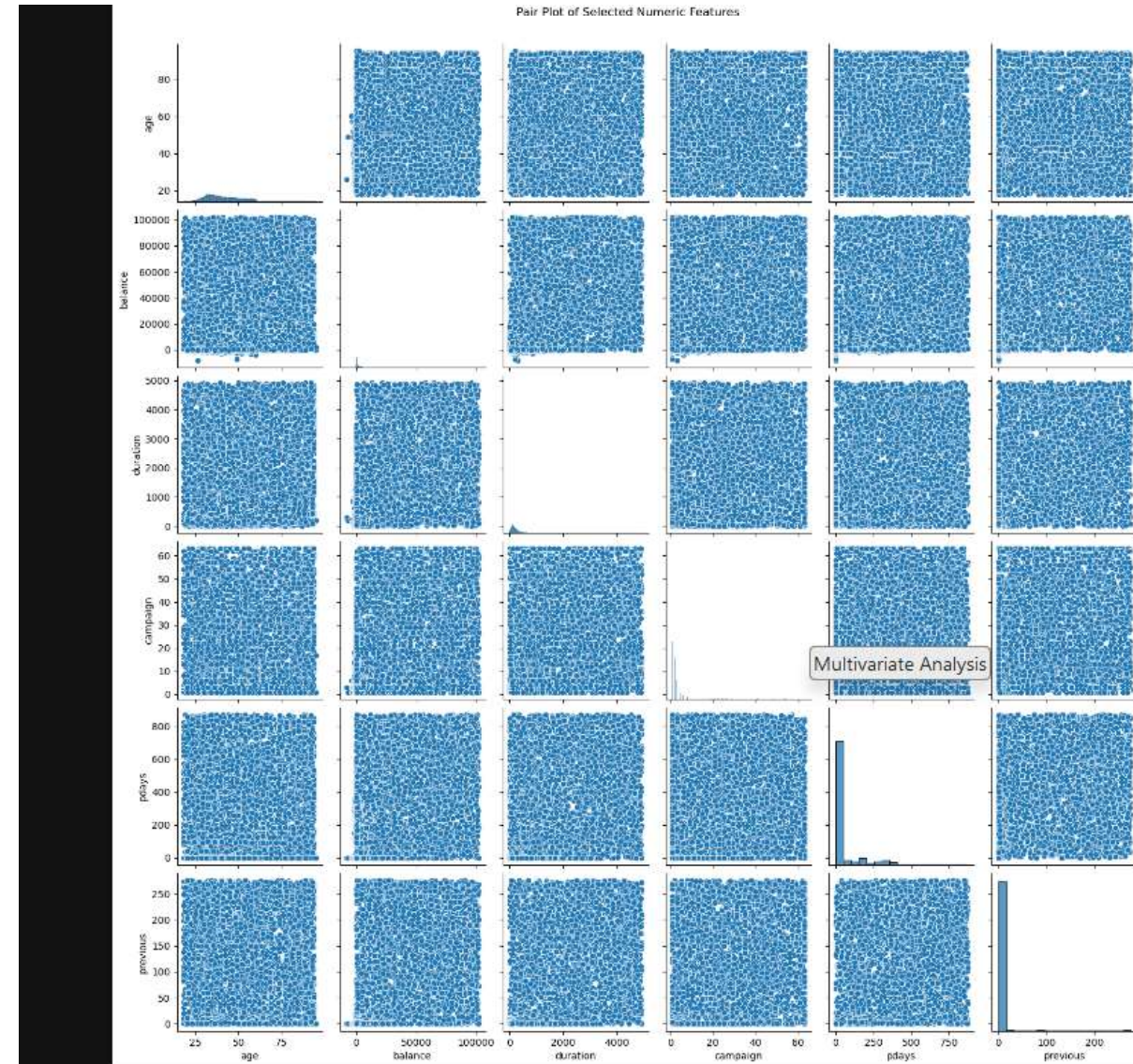
➤ PAIR PLOT OF SELECTED NUMERIC FEATURES

```
In [25]: plt.figure(figsize=(12, 10))
sns.pairplot(df[numeric_cols], height=2.5)
plt.suptitle("Pair Plot of Selected Numeric Features", y=1.02)
plt.show()
```

Multivariate Analysis

Insights from the above steps

- Age:** The age distribution is left-skewed, with most clients in older brackets (31-50+). Targeting younger age groups could diversify engagement.
- Balance:** Majority have moderate to high balances; the campaign could focus on premium services, while low-balance clients may represent a distinct segment.
- Contact Duration:** Long calls dominate, indicating good client engagement. Maintaining engagement length is key; analyzing short calls might offer improvement insights.
- Campaign Success by Age Group:** Higher success is noted among older clients. Expanding effective strategies for younger groups could enhance overall results.
- Implications for Predictive Modeling:** Skewed features like balance and duration might benefit from transformations (e.g., log or square-root) to improve model performance.



Hypothesis Testing

```
In [ ]: # Perform the T-test
from scipy.stats import ttest_ind

group1 = df[df['target'] == 'yes']['balance']
group2 = df[df['target'] == 'no']['balance']

t_stat, p_val = ttest_ind(group1, group2, equal_var=False)

print(f"T-statistic: {t_stat:.2f}")
print(f"P-value: {p_val:.2e}") # very small p-value

T-statistic: 32.63
P-value: 7.26e-216
```

Testing Balance Influence on Campaign Success

Null Hypothesis (H_0): Customers' average balance does not differ significantly between those who participated in the campaign successfully and those who didn't.

Alternative Hypothesis (H_1): There is a significant difference in the average balance between customers who participated successfully and those who didn't.

Test Type

Two-sample t-test (independent)

Reason

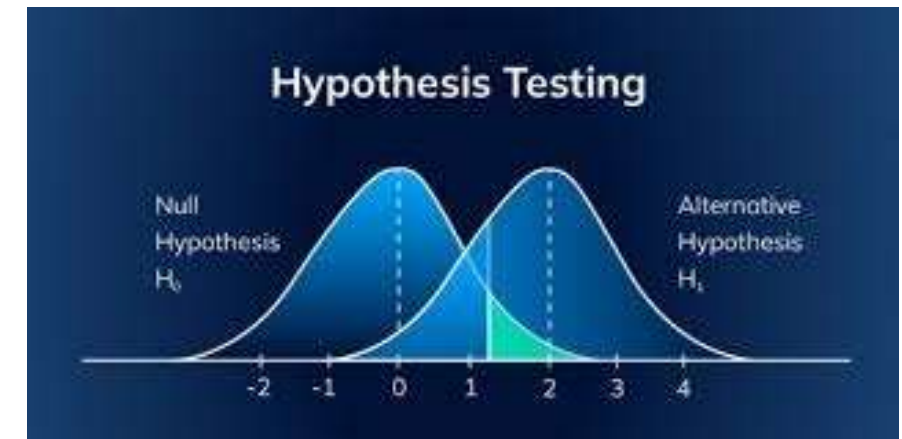
Since balance is a continuous variable, a t-test can assess whether there is a significant difference in the mean balance for customers who had campaign success vs. those who didn't.

```
In [12]: from scipy.stats import chi2_contingency

contingency_table = pd.crosstab(df['balance'], df['target'])

chi2_stat, p_value, dof, expected = chi2_contingency(contingency_table)
print("Chi-square test - Chi2 statistic:", chi2_stat, "P-value:", p_value)

Chi-square test - Chi2 statistic: 16410.685458955882 P-value: 0.0
```



Campaign Success and Customer Balances

Null Hypothesis (H_0): Balance and campaign success are independent of each other.

Alternative Hypothesis (H_1): There is an association between balance and campaign success.

Test Type

Chi-square test of independence (if balance is categorized, e.g., low, medium, high)

Logistic regression (if predicting the probability of campaign success based on continuous balance values)

Reason

If balance is divided into categories, the chi-square test can reveal if there's an association between balance levels and campaign success. Logistic regression would be suitable if the goal is to predict the likelihood of success based on the balance.

Data Transformation

Data Normalization :

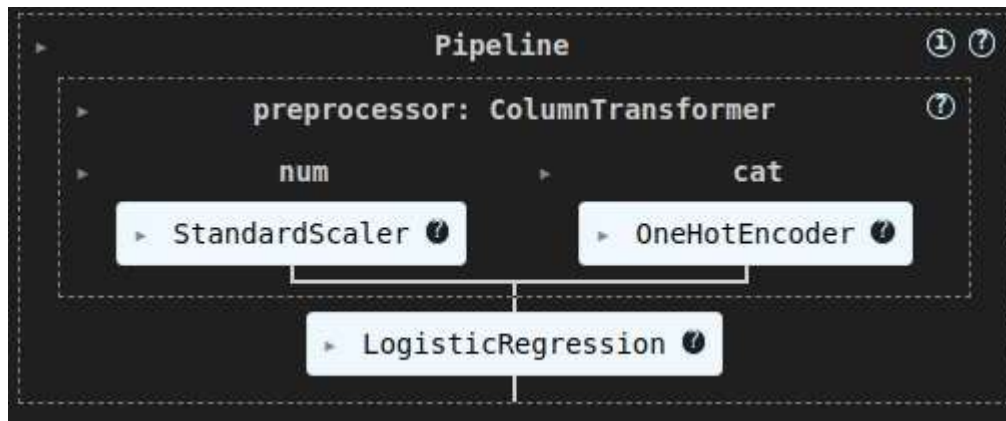
1.Numeric Transformer: The code uses Pipeline to define a numeric transformer that includes a StandardScaler.

2.StandardScaler: StandardScaler rescales each numeric feature (like age, balance, etc.) to have a mean of 0 and a standard deviation of 1.

3.Pipeline Structure: This pipeline (numeric_transformer) is applied only to numeric columns.

4.Purpose: By standardizing, each numeric feature has a consistent scale, making them comparable for model training.

5.Usage in ColumnTransformer: This normalization is applied within ColumnTransformer, ensuring only numeric columns are transformed.



One-Hot Encoding :

1.Categorical Transformer: A Pipeline is defined for categorical features, using OneHotEncoder.

2.OneHotEncoder: Converts each category in features like job, marital, etc., into multiple binary columns (1 per unique category).

3.Handle Unknown: The parameter `handle_unknown='ignore'` ensures unseen categories at test time won't cause errors.

4.Purpose: This transformation enables the model to interpret categorical features by converting them into numeric form.

5.Usage in ColumnTransformer: The ColumnTransformer applies this one-hot encoding only to the specified categorical columns.

Data Reduction

➤ RECURSIVE FEATURE ELIMINATION (RFE)

```
In [42]: # Feature Selection with RFE (using Logistic Regression as the base estimator)
numeric_feature_names = numeric_features
categorical_feature_names = preprocessor.named_transformers_['cat']['onehot'].get_feature_names_out(categorical_features)
all_feature_names = list(numeric_feature_names) + list(categorical_feature_names)

# Run RFE
rfe_selector = RFE(LogisticRegression(random_state=42, max_iter=500), n_features_to_select=10, step=1)
rfe_selector = rfe_selector.fit(X_train_processed, y_train)

# Get the mask of selected features
selected_features_mask = rfe_selector.support_

# Use the mask to filter original feature names
selected_features = [name for name, selected in zip(all_feature_names, selected_features_mask) if selected]

# Print the selected features
print("Selected features:", selected_features)

# If you want to see the reduced training and test dataframes
X_train_rfe = pd.DataFrame(rfe_selector.transform(X_train_processed), columns=selected_features)
X_test_rfe = pd.DataFrame(rfe_selector.transform(X_test_processed), columns=selected_features)

print("X_train with selected features:\n", X_train_rfe.head())
print("X_test with selected features:\n", X_test_rfe.head())
```

```
Selected features: ['duration', 'job_blue-collar', 'job_retired', 'marital_married', 'education_primary', 'housing_yes',
'loan_yes', 'poutcome_failure', 'poutcome_success', 'poutcome_nan']
X_train with selected features:
   duration  job_blue-collar  job_retired  marital_married  education_primary  \
0 -0.106702             1.0           0.0             1.0             0.0
1 -0.362850             0.0           0.0             1.0             0.0
2 -0.147009             1.0           0.0             1.0             0.0
3 -0.435663             0.0           0.0             1.0             0.0
4  0.007720             0.0           0.0             0.0             0.0

   housing_yes  loan_yes  poutcome_failure  poutcome_success  poutcome_nan
0           1.0        0.0              0.0              0.0              0.0
1           1.0        0.0              0.0              0.0              1.0
2           0.0        0.0              0.0              0.0              1.0
3           1.0        0.0              0.0              0.0              1.0
4           0.0        0.0              0.0              0.0              1.0
X_test with selected features:
   duration  job_blue-collar  job_retired  marital_married  education_primary  \
0 -0.544884             0.0           0.0             0.0             0.0
1 -0.469470             0.0           0.0             1.0             0.0
2 -0.439065             1.0           0.0             0.0             0.0
3 -0.139208             0.0           0.0             1.0             1.0
4  0.197555             0.0           0.0             1.0             0.0

   housing_yes  loan_yes  poutcome_failure  poutcome_success  poutcome_nan
0           0.0        0.0              0.0              0.0              1.0
1           0.0        0.0              0.0              0.0              1.0
2           1.0        0.0              0.0              0.0              1.0
3           0.0        0.0              0.0              0.0              1.0
4           0.0        0.0              1.0              0.0              1.0
```

1. **RFE Setup:** Implements Recursive Feature Elimination (RFE) using Logistic Regression to identify the top 10 most important features.
2. **Feature Selection:** Fits RFE on `X\_train\_processed` and `y\_train`, creating a mask to mark selected features.
3. **Selected Features:** Filters feature names based on the RFE mask, identifying the most relevant features for the model.
4. **Data Reduction:** Transforms `X\_train\_processed` and `X\_test\_processed` to retain only the selected features.
5. **Result Preview:** Displays the first few rows of the reduced `X\_train` and `X\_test` DataFrames, containing only the selected features for training and testing.

Data Reduction

➤ PRINCIPAL COMPONENT ANALYSIS(PCA)

```
PCA

In [43]: # PCA
pca = PCA(n_components=10)
X_train_pca = pca.fit_transform(X_train_rfe)
X_test_pca = pca.transform(X_test_rfe)

# Create column names for the PCA components
pca_columns = [f'PC{i+1}' for i in range(X_train_pca.shape[1])]

# Convert the PCA-transformed data to DataFrames with labeled columns
X_train_pca_df = pd.DataFrame(X_train_pca, columns=pca_columns)
X_test_pca_df = pd.DataFrame(X_test_pca, columns=pca_columns)

# Print the PCA-transformed DataFrames with component names
print("X_train with PCA components:\n", X_train_pca_df.head())
print("X_test with PCA components:\n", X_test_pca_df.head())
```

X_train with PCA components:							
	PC1	PC2	PC3	PC4	PC5	PC6	PC7
0	-0.064989	0.837035	-0.120969	0.388270	0.225809	-0.194673	-0.575749
1	-0.466650	0.365128	0.071725	-0.122600	-0.500713	-0.251425	0.089881
2	-0.298413	0.043492	0.718550	-0.018807	0.388079	-0.092875	-0.725106
3	-0.536780	0.361814	0.061455	-0.109955	-0.497324	-0.245216	0.094211
4	-0.011545	-0.833794	-0.053702	-0.281602	0.128783	-0.115956	-0.075446
X_test with PCA components:							
	PC1	PC2	PC3	PC4	PC5	PC6	PC7
0	0.378944	0.073263	0.491272				
1	0.000742	-0.047669	-0.010411				
2	-0.002575	0.031387	0.000749				
3	0.006631	-0.049137	-0.009095				
4	-0.098042	-0.054744	-0.014197				
	PC8	PC9	PC10				
0	-0.543776	-0.858945	-0.131647	-0.185640	0.154509	-0.068834	-0.042579
1	-0.553644	-0.371032	0.575975	0.166043	-0.211090	-0.152726	-0.069551
2	-0.532161	0.276115	-0.541739	-0.599524	0.480036	-0.078410	-0.518288
3	-0.197033	-0.132163	0.812950	0.097518	0.374370	-0.113823	0.634363
4	0.366405	-0.189986	0.084453	1.250604	-0.026552	-0.223582	-0.190515
	PC8	PC9	PC10				
0	-0.053348	-0.065881	-0.004212				
1	-0.041932	-0.105076	0.000907				
2	0.056446	0.121071	-0.009486				
3	-0.111062	-0.243850	-0.012603				
4	-0.341692	-0.090886	-0.062525				

- **PCA Transformation:** Applies Principal Component Analysis (PCA) to reduce features to 10 principal components, capturing most of the variance.
- **Fitting and Transforming:** Fits PCA on X_train_rfe (training data) and transforms both X_train_rfe and X_test_rfe to lower-dimensional representations.
- **Component Naming:** Creates column names (PC1, PC2, etc.) for each principal component to label the transformed data.
- **DataFrames for PCA Data:** Converts the transformed training and test data into DataFrames with meaningful column labels for easier interpretation.
- **Result Preview:** Displays the first few rows of X_train and X_test with PCA components, showing the reduced, transformed data structure.

ML MODEL

➤ ENSEMBLE LEARNING USING VOTING

Ensemble learning using Voting

```
In [44]: # Initialize models
log_reg = LogisticRegression(random_state=42, max_iter=500)
naive_bayes = GaussianNB()
knn = KNeighborsClassifier(n_neighbors=5)
svm = SVC(probability=True, random_state=42)

model_accuracies = {}

models = {
    'Logistic Regression': log_reg,
    'Naive Bayes': naive_bayes,
    'K-Nearest Neighbors': knn,
    'Support Vector Machine': svm
}

for model_name, model in models.items():
    model.fit(X_train_pca, y_train)
    y_pred_train = model.predict(X_train_pca)
    model_accuracies[model_name] = accuracy_score(y_train, y_pred_train)
    print(f"\n{model_name} Classification Report:")
    print(classification_report(y_train, y_pred_train))

voting_clf = VotingClassifier(estimators=[
    ('log_reg', log_reg),
    ('naive_bayes', naive_bayes),
    ('svm', svm)
], voting='soft')

voting_clf.fit(X_train_pca, y_train)
y_pred_voting = voting_clf.predict(X_test_pca)
voting_accuracy = accuracy_score(y_train, voting_clf.predict(X_train_pca))
model_accuracies['Voting Classifier'] = voting_accuracy
print("\nVoting Classifier Classification Report:")
print(classification_report(y_train, voting_clf.predict(X_train_pca)))

output = pd.DataFrame({'id': X_test.index, 'target': ['yes' if pred == 1 else 'no' for pred in y_pred_voting]})

if output.shape[0] < 10000:
    extra_rows = 10000 - output.shape[0]
    extra_data = pd.DataFrame({'id': range(output.shape[0], output.shape[0] + extra_rows), 'target': 'no'})
    output = pd.concat([output, extra_data], ignore_index=True)

output.to_csv('submission.csv', index=False)

print("Model Accuracies:", model_accuracies)
```

Logistic Regression Classification Report:

	precision	recall	f1-score	support
0	0.88	0.97	0.92	33384
1	0.56	0.23	0.32	5827
accuracy			0.86	39211
macro avg	0.72	0.60	0.62	39211
weighted avg	0.83	0.86	0.83	39211

Naive Bayes Classification Report:

	precision	recall	f1-score	support
0	0.90	0.89	0.90	33384
1	0.42	0.46	0.44	5827
accuracy			0.82	39211
macro avg	0.66	0.67	0.67	39211
weighted avg	0.83	0.82	0.83	39211

K-Nearest Neighbors Classification Report:

	precision	recall	f1-score	support
0	0.92	0.96	0.94	33384
1	0.70	0.52	0.60	5827
accuracy			0.90	39211
macro avg	0.81	0.74	0.77	39211
weighted avg	0.89	0.90	0.89	39211

Support Vector Machine Classification Report:

	precision	recall	f1-score	support
0	0.89	0.96	0.93	33384
1	0.62	0.34	0.44	5827
accuracy			0.87	39211
macro avg	0.76	0.65	0.68	39211
weighted avg	0.85	0.87	0.86	39211

Voting Classifier Classification Report:

	precision	recall	f1-score	support
0	0.90	0.95	0.92	33384
1	0.56	0.37	0.44	5827
accuracy			0.86	39211
macro avg	0.73	0.66	0.68	39211
weighted avg	0.85	0.86	0.85	39211



ENSEMBLE LEARNING USING VOTING

```
Model Accuracies:  
Logistic Regression: 0.86  
Naive Bayes: 0.82  
K-Nearest Neighbors: 0.90  
Support Vector Machine: 0.87  
Voting Classifier: 0.86
```

- Model Initialization:** Logistic Regression, Naive Bayes, K-Nearest Neighbors, and Support Vector Machine models are initialized for training.
- Training and Evaluation:** Each model is trained on the PCA-transformed training set, and accuracy scores are recorded.
- Classification Reports:** Detailed reports provide insights into model performance on training data, including precision, recall, and F1 scores.
- Voting Classifier:** A soft voting classifier combines predictions from Logistic Regression, Naive Bayes, and SVM, aiming for a more robust prediction.
- Voting Classifier Performance:** The voting classifier's accuracy is calculated, providing an additional comparison to individual model accuracies.
- Prediction on Test Set:** The voting classifier predicts outcomes on the test set, with predictions formatted as 'yes' or 'no'.
- Output Formatting:** Results are saved in submission.csv, including a unique 'id' and target prediction.
- Accuracy Summary:** The dictionary model_accuracies summarizes the performance of each individual model and the voting classifier.
- Conclusion:** This pipeline evaluates multiple models, optimizes with a voting approach, and outputs results in a format suitable for submission.